Convoy: shipment tracking prototype

Convoy gives a manufacturer's customers one place to see where an order is and when it arrives, and its ops team a short list of what to chase today. It is a clickable prototype on mocked data, with the AI simulated by rules.

The cut I chose and why

Two roles, and only the screens that answer one question each. The customer asks "is my order on time, and when does it arrive?": a shipment list, a detail page and a notification feed. Ops asks "which shipments do I chase first?": Today, an exceptions queue, a portfolio table and a detail page with the raw operator feed beside the milestones.

Ops covers two of the four sites by origin or destination; one shipment sits outside on purpose, so its URL says "Shipment not found" and search returns nothing. Nordvik, the hero customer, receives in Norway, outside the EU customs union: real import customs at Oslo and a Norwegian broker posting Norwegian statuses. Sea arrivals inside the EU only get "Port formalities (Union status check)" and a "Proof of Union status (T2L)"; a customs declaration exists only outside the union.

One palette (manifest grey, ink, harbour blue for interaction, cleared green, amber, signal red), Barlow with tabular numbers, white panels with graded corners (12 px panels, 8 px controls, pills for status, risk, chips, stamps and the role switcher) and no shadows except on floating layers. The route timeline is the one bold element: a 6 px rounded ink track where operators confirmed, dashes where planned, an amber band under the span an estimate moved, ring nodes and a legend. An earlier flat "ledger" look read as a generic broadsheet in review, so I reworked it.

Left out, and why:

- A map. A good one costs hours, a cheap one looks generic; the timeline carries the same information.
- Real operator integrations, document upload, analytics, dark mode, a translated UI. The brief is frontend only, and none of these changes the answer to the two questions.
- Automated tests. I spent the hours on the product and checked by hand: lint, build, both roles at 1440 and 360 px, keyboard, reduced motion. A real gap.

Where and how the AI adds value

Seven features, each placed where someone needs it.

Feature	Where	Based on
Exception management	Ops Today, exceptions queue	Delay, customs hold, silence, documents, port incidents; one proposed action per row.
Arrival estimates	Shipment detail, both roles	Confirmed delays plus typical customs time, with confidence and reason.
Consolidation	Timeline, ops detail	Seven vocabularies ("Gate-in", "VSL DEP", "Fortollet") mapped to one milestone set, raw feed alongside.
Daily summary	Ops Today	Live counts and ranked next actions, recomputed after every action.

Feature	Where	Based on
Natural-language search	Ops portfolio	"shipments to France this week running late" becomes three chips and a filtered table; leftover words are marked ignored; one control clears all.
Proactive notices	Exceptions queue, customer banner, feed	Order, route, estimate, confidence, cause; ops edits before sending.
Drafting in the recipient's language	Message dialogs	Customer account language or operator language; French for Rhone, reason shown.

Outputs are branded Convoy, not "AI": the user should trust the product and see the facts it used. Every output carries a stamp that explains what it was built from and a "Why" line naming those facts. The interface never says "prototype" or "simulated"; the README and this document do.

A confirmed date and a Convoy date never look the same: operator events solid, planned legs dashed, the span an estimate moved on the amber band. One `Estimate` component is the only code that renders an arrival; if nothing moved the date it says "Planned", not "Estimated". An unknown status shows as "Not mapped"; a quiet feed shows the customer "Awaiting update". I would rather show a gap than guess.

The live feed plays this out: seven scripted operator events, one every four seconds, with pause, resume and reset, on the demo clock. Each lands with a highlight that fades, a toast linking to the order and the header's "Last updated" time. When Nordhaul posts its own ETA on order 12466, it is an operator fact: solid, labelled "Operator ETA", and the superseded Convoy estimate moves into "ETA history".

Each dialog shows a "To" line with recipient and channel: email for operators, Convoy inbox plus email for customers. Documents follow ownership: the manufacturer's own (invoice, packing list, T2L) are sent with the ERP file attached (a fake picker); the carrier's bill of lading or CMR and the broker's customs declaration are requested. The customer never gets a document request, and downloads existing documents as one-page PDFs built in the browser. Notices are capped at 600 characters and clamped to three lines for the customer.

The switch only goes towards English, the trade's shared language: "Write in English instead" regenerates the draft, "Back to French" undoes it, and an English draft offers no switch, since with many languages Convoy could not know which one to offer.

Technical approach

Store facts, derive judgments. React 19, Vite, react-router for real URLs, plain CSS with custom-property tokens; no backend, UI kit or state library. The store (context plus reducer) holds operator events, the plan, documents and what ops did. Risk, estimates, summary, search and drafts are pure functions in `src/ai/`, run through selectors, so an ops action reaches the customer view with no extra wiring.

Risk weights are plain numbers (+4 for a customs hold, +3 for 24 hours of silence, +2 per day of slip capped at +6): same inputs, same ranking. The clock is fixed at 13 Oct 2026, 14:30, and the live feed stamps its events on it. 24 shipments, seven operators, four sites, identical on any machine.

In production an LLM maps free-text statuses to milestones against a schema; rules stay as fallback and low confidence lands in "Not mapped". Estimates come from a model on lane history and port feeds, with thresholds deciding between a date, a range or "no update yet". A model drafts notices and a human approves each one. Search returns the same filter object the dropdowns use, so chips stay inspectable. A labelled set of real statuses and past delays scores every change before release. Events arrive by webhook, polling as fallback.

Assumptions, limitations and next steps

Assumptions: one customer and one ops persona are enough to show scoping; an English UI, metric units and EUR; operators that post timestamped events; a local demo, so no deployment or auth.

Limitations:

- Reload resets all data.
- The live feed is a seven-event script.
- Search understands a fixed set of phrases.
- Risk and estimates are rules, not models.
- Drafts in English and French only.
- No auth, no tests.

Next steps, in order:

1. Real events behind a thin API with webhooks.
2. An evaluation set for the status mapper, before any LLM touches customer-facing text.
3. Tests for the risk and search rules, the cheapest to cover.
4. Notification preferences, document upload and a map, once I know what customers ask for.

I used AI coding tools (Claude Code) throughout. I made the scope, product and visual decisions and reviewed what it produced.